TradeNexa API Documentation

REST API Reference for Frontend Developers — Authentication, Categories, Products & Brands

Version: 1.0 | **Base URL:** `{APP_URL}/api/v1`

Example: `http://localhost:3000/api/v1`

Content-Type: `application/json` (unless noted as `multipart/form-data`)

Media URL base: `{APP_URL}/uploads/` — image fields return full URLs when `APP_URL` is configured.

Table of Contents

1. General Conventions
2. Authentication Module
3. Categories & Subcategories
4. Products Module
5. Brands Module
6. Appendix

1. General Conventions

1.1 Response Envelope

All successful responses follow this structure:

```
{
  "success": true,
  "message": "Human-readable message",
  "data": { }
}
```

1.2 Error Response

```
{
  "success": false,
  "message": "Error summary",
  "errors": [
    { "field": "email", "message": "Invalid email" }
  ]
}
```

HTTP Code	Meaning
400	Validation error or bad request
401	Missing or invalid access token
403	Forbidden — insufficient role permissions
404	Resource not found
409	Conflict (duplicate entry, referenced record)
500	Internal server error

1.3 Authentication Header

Protected endpoints require a JWT access token:

```
Authorization: Bearer <access_token>
```

1.4 Pagination

List endpoints return paginated data inside `data`:

```
{
  "success": true,
  "message": "...",
  "data": {
    "results": [ ],
    "pagination": {
      "total": 100,
      "page": 1,
      "limit": 10,
      "totalPages": 10
    }
  }
}
```

Query Param	Type	Default	Description
page	integer	1	Page number (min: 1)
limit	integer	10	Items per page (min: 1, max: 100)

Default sort order: All list APIs return results ordered by `id DESC` (newest first).

1.5 File Uploads (multipart/form-data)

Several write endpoints accept file uploads. Rules:

- Use `Content-Type: multipart/form-data`
- Send images as **files**, not URL strings (URL text in body is rejected)
- Allowed formats: JPEG, PNG, WEBP, GIF — max 5 MB per file
- Boolean fields in form-data: send as string `"true"` or `"false"`

2. Authentication Module

2.1 User Authentication Flow (Mobile OTP)

1. `POST /auth/send-otp` — send OTP, receive `firebase_verification_id`
2. `POST /auth/verify-otp` — verify OTP
3. If `is_registered: false` → `POST /auth/register` with registration token
4. If `is_registered: true` → use `access_token` and `refresh_token` directly
5. Complete profile via `PUT /auth/profile` (sets `is_completed_profile: true`)

POST **`/auth/send-otp`** Public

Send OTP to a mobile number via Firebase.

Request Body (JSON)

Field	Type	Required	Description
mobile_number	string	Yes	Format: `+919876543210` (country code + 10-digit Indian mobile)
recaptcha_token	string	No	reCAPTCHA token if enabled

Response `data`

```
{
  "firebase_verification_id": "firebase-session-id",
  "mobile_number": "+919876543210"
}
```

POST `/auth/verify-otp` Public

Verify OTP. Returns full auth tokens if user exists, or a registration token for new users.

Request Body (JSON)

Field	Type	Required	Description
mobile_number	string	Yes	Same number used in send-otp
otp	string	Yes	4–8 digit OTP code
firebase_verification_id	string	Yes	From send-otp response
device	object	No	`{ device_type: "android"|"ios"|"web", device_token: "..." }`

Response — Existing User

```
{
  "is_registered": true,
  "is_completed_profile": false,
  "user": { "id": 1, "full_name": "...", "email": "...", "role": "buyer", ...
},
  "access_token": "eyJ...",
  "refresh_token": "eyJ..."
}
```

Response — New User (not registered)

```
{
  "is_registered": false,
  "is_completed_profile": false,
  "mobile_number": "+919876543210",
  "access_token": "eyJ..."
}
```

> Use the registration `access_token` as `Authorization: Bearer` when calling `POST /auth/register`.

POST /auth/resend-otp Public

Field	Type	Required
mobile_number	string	Yes
firebase_verification_id	string	Yes
recaptcha_token	string	No

POST /auth/register Registration Token

Register a new user after OTP verification.

Headers

```
Authorization: Bearer <registration_access_token>
```

Request Body (JSON)

Field	Type	Required	Description
mobile_number	string	Yes	Verified mobile number
full_name	string	Yes	2–100 characters
email	string	Yes	Valid unique email
role_id	integer	Yes	Role ID (buyer, seller, buyer_seller)
business_type_id	integer	Yes	Must match selected role
language_id	integer	No	Defaults to English if omitted
device	object	No	Push notification device info

Response

Same as verify-otp for registered user (tokens + user object).

POST `/auth/refresh-token` Public

Field	Type	Required
refresh_token	string	Yes

Response `data`

```
{ "access_token": "eyJ...", "refresh_token": "eyJ..." }
```

POST `/auth/logout` Auth Required

Field	Type	Required
refresh_token	string	No

2.2 User Profile

GET `/auth/profile` Auth Required

Returns full profile including company details, address, and image URLs.

PUT `/auth/profile` Auth Required

Content-Type: `multipart/form-data`

Fields vary by role. Sets `is_completed_profile = true` on success.

Image Fields (file upload — optional)

Field	Roles
profile_image	buyer, buyer_seller
company_logo	seller, buyer_seller
company_banner	seller, buyer_seller

Buyer Required Text Fields

`company_name`, `industry`, `address_line_1`, `country`

Seller Required Text Fields

`company_name`, `gst_number`, `pan_number`, `business_description`

Buyer+Seller Required Text Fields

All buyer + seller fields combined.

> Do not send `mobile_number`, `role_id`, or image fields as URL text in the body.

DELETE `/auth/profile` Auth Required

Soft-deletes the authenticated user account.

2.3 Admin Panel Authentication

POST `/admin/auth/login` Public

Request Body (JSON)

Field	Type	Required
email	string	Yes
password	string	Yes

Response `data`

```
{
  "user": { "id": 1, "full_name": "...", "email": "...", "role": "admin" },
  "access_token": "eyJ...",
  "refresh_token": "eyJ..."
}
```

Allowed roles: `admin`, `super_admin`, `supporter`

POST `/admin/auth/users` Admin / Super Admin

Request Body (JSON)

Field	Type	Required	Description
full_name	string	Yes	2–100 chars
email	string	Yes	Unique email
password	string	Yes	8–128 chars, upper, lower, number, special char
role_id	integer	Yes	Admin panel role ID

3. Categories & Subcategories Module

Hierarchy: **Main Category** (`parent_id = null`) → **Subcategory** (`parent_id` set). Products link to `subcategory_id` .

3.1 Main Categories — Read

GET `/categories` Public

Query Parameters

Param	Type	Description
page, limit	integer	Pagination
q	string	Search by name
is_active	boolean	Default: `true`

Result item fields

`id` , `name` , `slug` , `icon` , `image` , `is_active` , `subcategory_count` , `product_count`

GET `/categories/:id` Public

Returns category detail with nested `subcategories` array.

3.2 Subcategories — Read

GET `/categories/:categoryId/subcategories` Public

Query Parameters

Param	Type	Description
page, limit	integer	Pagination
q	string	Search by name
is_active	boolean	Default: `true`

GET `/categories/:categoryId/subcategories/:id` Public

Single subcategory with `category_id`, `category_name`, `product_count`, `icon`, `image`, `slug`.

3.3 Main Categories — Write (Admin)

POST `/categories` Admin

Content-Type: `multipart/form-data`

Field	Type	Required
name	text	Yes
slug	text	No
is_active	text	No — `true` / `false`
icon	file	No
image	file	No

PUT `/categories/:id` Admin

Same as create — all fields optional on update. `multipart/form-data`.

DELETE `/categories/:id` Admin

Fails if category has active subcategories.

3.4 Subcategories — Write (Admin)

POST `/categories/:categoryId/subcategories` Admin

`multipart/form-data` — same fields as category create. Slug stored as `{parent-slug}-{child-slug}`.

PUT `/categories/:categoryId/subcategories/:id` Admin

`multipart/form-data` — all fields optional.

DELETE `/categories/:categoryId/subcategories/:id` Admin

Fails if subcategory is linked to products.

4. Products Module

Products are linked to **subcategories** (not main categories). Thumbnail is uploaded as a file.

4.1 Read Endpoints

GET `/products` Public

Query Parameters

Param	Type	Description
page, limit	integer	Pagination
q	string	Search by product name
category_id	integer	Filter by main category (all its subcategories)
subcategory_id	integer	Filter by specific subcategory
brand_id	integer	Filter by brand
min_price, max_price	number	Price range filter
is_active	boolean	Default: `true`
sort_by	string	Optional: `price_asc`, `price_desc`, `rating`. Default: `id DESC`

GET `/products/trending` Public

Returns products where `is_trending = true`. Paginated. Ordered by `id DESC`.

Formatted result fields

`id`, `name`, `thumbnail`, `price`, `currency`, `moq`, `unit`, `supplier_name`, `verified`, `rating`, `city`, `state`

GET `/products/recommended` Public

Query Parameters

Param	Type	Required	Description
subcategory_id	integer	Yes	Filter products in this subcategory
product_id	integer	No	Exclude this product (e.g. current product detail page)
page, limit	integer	No	Pagination

Formatted result fields

`id` , `name` , `thumbnail` , `price` , `moq` , `unit` , `supplier_name` , `verified`

GET `/products/:id` Public

Full product detail including `category_name` , `subcategory_id` , `subcategory_name` , `brand_name` , supplier location.

4.2 Write Endpoints (Seller / Buyer+Seller / Admin)

POST `/products` Seller / Admin

Content-Type: `multipart/form-data`

Field	Type	Required	Description
name	text	Yes	2–200 characters
price	text/number	Yes	Positive number
supplier_id	text/integer	Yes	Supplier user ID
subcategory_id	text/integer	Yes	Must be a subcategory ID
thumbnail	file	No	Product image file
slug	text	No	URL-safe slug
currency	text	No	Default: `INR`
moq	text/integer	No	Default: 1
unit	text	No	Default: `pcs`
brand_id	text/integer	No	Brand ID
is_trending	text	No	`true` / `false`
rating	text/number	No	0–5
is_active	text	No	`true` / `false`

PUT `/products/:id` Seller / Admin

`multipart/form-data` — all fields optional on update.

DELETE `/products/:id` Seller / Admin

Soft-deletes the product.

5. Brands Module

GET `/brands` Public

Query Parameters

Param	Type	Description
page, limit	integer	Pagination
q	string	Search by brand name
is_popular	boolean	`true` — popular brands only
is_active	boolean	Default: `true`

Result item fields

`id`, `name`, `logo` (full URL), `is_popular`, `is_active`

> Use `GET /brands?is_popular=true` instead of a separate popular-brands endpoint.

GET `/brands/:id` Public

Single brand detail.

POST `/brands` Admin

Content-Type: `multipart/form-data`

Field	Type	Required
name	text	Yes
logo	file	No
is_popular	text	No — `true` / `false`
is_active	text	No — `true` / `false`

PUT `/brands/:id` Admin

`multipart/form-data` — all fields optional.

DELETE `/brands/:id` Admin

Soft-deletes the brand.

6. Appendix

6.1 Role Codes

Code	Description
buyer	Marketplace buyer
seller	Marketplace seller / supplier
buyer_seller	Both buyer and seller
admin	Platform administrator
super_admin	Full platform access
supporter	Customer support staff

6.2 Quick Reference — Endpoint Summary

Method	Endpoint	Auth
POST	/auth/send-otp	Public
POST	/auth/verify-otp	Public
POST	/auth/register	Registration token
POST	/auth/refresh-token	Public
GET/PUT/DELETE	/auth/profile	User token
POST	/admin/auth/login	Public
POST	/admin/auth/users	Admin token
GET	/categories	Public
GET	/categories/:id	Public
GET	/categories/:categoryId/subcategories	Public
GET	/categories/:categoryId/subcategories/:id	Public
POST/PUT/DELETE	/categories/*	Admin
GET	/products, /products/trending, /products/recommended	Public
GET	/products/:id	Public
POST/PUT/DELETE	/products/*	Seller / Admin
GET	/brands, /brands/:id	Public
POST/PUT/DELETE	/brands/*	Admin